

Задача 1. Лучшее совмещение

Гавриленко Матвей, 2 курс, 2 группа

Постановка задачи

Дано n строк одинаковой длины m . Нужно найти такую строку T максимальной длины, которая может быть получена как префикс каждой из циклически сдвинутых строк, и указать для каждой строки величину сдвига (позицию начала T).

Идея решения

Решение основано на трёх ключевых компонентах:

- хеширование строк;
- бинарный поиск по длине общей подстроки;
- проверка достижимости заданной длины.

1. Хеширование строк

Для эффективного сравнения подстрок используются двойные полиномиальные хеши (обратные). Каждая строка преобразуется в числовое представление — пару хешей на разных основаниях и модулях. Это позволяет получать хеш любой подстроки за $O(1)$ после предварительной обработки (вычисления степеней основания). Два независимых хеша исключают вероятность коллизий.

Замечание: Для работы с циклическими сдвигами каждая строка удваивается ($s[i] + s[i]$). После этого любой циклический сдвиг исходной строки длины m превращается в обычную подстроку удвоенной строки.

2. Бинарный поиск

Используется свойство монотонности: если существует общая подстрока длины L , то существует и общая подстрока любой меньшей длины (достаточно взять префикс). Это позволяет применить бинарный поиск для нахождения максимальной длины искомой подстроки.

Поиск ведётся по длине от 0 до m . На каждой итерации проверяется, существует ли общая подстрока длины mid . Двигаем границы в зависимости от результата проверки.

3. Проверка заданной длины (`canAchieveLcp`)

Для фиксированной длины len необходимо ответить: существует ли строка, которая встречается во всех n строках как подстрока длины len при каком-то циклическом сдвиге.

Алгоритм проверки:

1. Для первой строки вычисляем хеши всех возможных подстрок длины len (все циклические сдвиги) и помещаем их в множество *common* (`unordered_map`).
2. Для каждой следующей строки:
 - Вычисляем все её хеши длины len ;
 - Оставляем в *common* только те хеши, которые присутствуют и в текущей строке (пересечение множеств);

Замечание: для каждой строки хранится только первое вхождение каждого хеша.

3. Если после обработки всех строк *common* не пуст — искомая подстрока существует; выбираем любой подходящий хеш.
4. Для найденного хеша определяем позицию начала в каждой строке (первое вхождение).

4. Восстановление ответа

После того как нашли максимальную длину *bestLen*, выполняется финальная проверка для этой длины, которая даёт конкретные позиции сдвигов для каждой строки. Позиции выводятся в 1-индексации (прибавляется 1).

Корректность алгоритма

Лемма 1 (Монотонность). *Если существует общая подстрока длины L , то для любого $L' \leq L$ существует общая подстрока длины L' .*

Доказательство. Пусть T — искомая подстрока длины L , и для каждой строки s_i она начинается с позиции p_i после циклического сдвига. Тогда первые L' символов T образуют строку T' длины L' , которая также является префиксом циклического сдвига каждой строки s_i (начинается с той же позиции p_i). $\square$

Лемма 2 (Корректность проверки длины). *Для фиксированного len функция `canAchieveLcp(len)` возвращает `true` тогда и только тогда, когда существует строка T длины len , являющаяся префиксом некоторого циклического сдвига каждой из строк s_i .*

Доказательство.

- $\Rightarrow$ Если функция вернула `true`, то после обработки всех строк множество *common* непусто. Выбранный хеш по построению присутствует в каждой строке как хеш подстроки длины len , начинающейся с некоторой позиции. Следовательно, соответствующая строка T существует.
- $\Leftarrow$ Если существует строка T длины len , то её хеш h является элементом множества *common*, и на каждом шаге пересечения он сохраняется, поскольку содержится в каждой строке. Поэтому после обработки всех строк $h \in common$, и функция вернёт `true`. $\square$

Теорема 1 (Корректность основного алгоритма). *Предложенный алгоритм находит максимальную длину $L_{\max}$ и соответствующие сдвиги для всех строк за конечное время.*

Доказательство. Бинарный поиск, опираясь на лемму о монотонности, находит наибольшее L , для которого `canAchieveLcp(L)` возвращает `true`. Существование хотя бы одного такого L гарантировано, так как $L = 0$ всегда достижимо. После завершения бинарного поиска выполняется финальный вызов `canAchieveLcp(Lmax)`, который возвращает конкретные позиции сдвигов для каждой строки. По построению эти сдвиги соответствуют одной и той же строке T длины $L_{\max}$. $\square$

Асимптотическая сложность

Сложность по времени

- **Предобработка:** $O(n \cdot t)$ — построение хешей всех удвоенных строк и вычисление степеней оснований.
- **Бинарный поиск:** $O(\log t)$ итераций.
- **Проверка одной длины:** $O(n \cdot t)$ — для каждой строки проходим по всем t сдвигам и выполняем операции с хеш-таблицами за $O(1)$ в среднем.

Итоговая сложность по времени: $O(n \cdot t \cdot \log t)$.

Сложность по памяти

- Хранение удвоенных строк: $O(n \cdot t)$.
- Префиксные хеши: $O(n \cdot t)$.
- Степени оснований: $O(t)$.
- Временные хеш-таблицы: $O(t)$.

Итоговая сложность по памяти: $O(n \cdot t)$.

Вывод

Разработанный алгоритм за время $O(n \cdot t \cdot \log t)$ и память $O(n \cdot t)$ корректно находит максимальную длину общей префиксной подстроки для циклических сдвигов n строк длины t .